

Critical Review: Partially Correct, But Missing Key Details

Let me grade your answers and correct the misconceptions.

Question 1: Sensitivity List MOSTLY CORRECT

Part A: CORRECT

"bcs we used reset as a asynchronous input that means it will work even if clk is on falling or rising without being dependent on en and clk."

Good. You understand asynchronous means independent of clock.

Part B: CORRECT

"reset will still work but it will synchronous input, i.e it will work only when the clk is on rising edge"

Correct. If we removed or `posedge reset` from sensitivity list, reset would only be checked at `posedge clk`, making it synchronous.

Part C: CORRECT

"asynchronous reset means it is not dependent on clk or en."

Correct. Asynchronous = independent of clock. But I'll add: it's still checked in the priority order inside the always block.

Question 2: Priority Logic WRONG

Part A: CORRECT

"bcs reset has the highest priority."

Correct.

Part B: COMPLETELY WRONG

"the priority order is like this: reset > en > clk > d"

This is WRONG. Let me correct your fundamental misconception:

Clock is NOT part of the priority chain!

The correct priority is: **reset > en > hold**

Here's why you're confused:

```
always @(posedge clk or posedge reset) begin // ← Sensitivity list (triggers)
    if (reset) // ← Priority 1
        q <= 0;
```

```

    else if (en)      // ← Priority 2
        q <= d;
    else              // ← Priority 3 (default)
        q <= q;
end

```

Sensitivity List vs Priority Logic:

- **Sensitivity list** (`@(posedge clk or posedge reset)`): Defines WHEN the block runs
- **Priority logic** (`if-else` chain): Defines WHAT happens when block runs

Clock (clk) is a TRIGGER, not a priority level. Data (d) is an INPUT, not a priority level.

The always block runs when:

- Clock rises (`posedge clk`), OR
- Reset rises (`posedge reset`)

Once triggered, the priority is:

1. Is `reset=1`? → Force `q=0`
2. Else, is `en=1`? → Capture `d`
3. Else → Hold current `q`

Part C: ❌ WRONG

"ntg will change."

This is COMPLETELY WRONG. If we swap the order:

// WRONG ORDER:

```

if (en)
    q <= d;
else if (reset)
    q <= 0;
else
    q <= q;

```

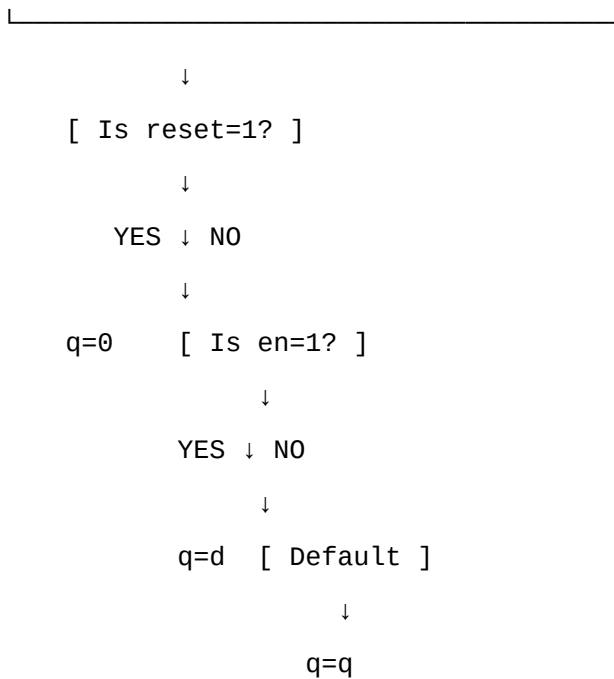
What breaks:

- When `en=1` AND `reset=1` simultaneously:
 - **Wrong code:** Checks `en` first → captures `d` → IGNORES `reset`!
 - **Correct code:** Checks `reset` first → forces 0 → ignores `en`

Reset would NO LONGER override enable! This is a critical bug.

Correct Priority Diagram:

Always Block Execution Decision



Question 3: Asynchronous Behavior CORRECT

Part A: CORRECT

"no."

Correct. Reset does NOT wait for clock edge.

Part B: CORRECT

"instantly."

Correct. At time 73ns (between clock edges), q would go to 0 immediately.

Part C: CORRECT

"bcs it does not wait for edge signal. unlike synchronous."

Correct. This is the definition of asynchronous.

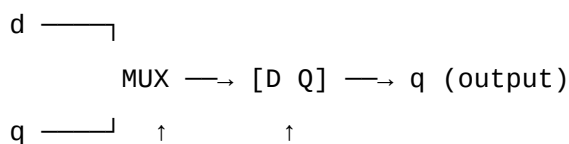
Question 4: Self-Assignment - YOU DIDN'T ANSWER THIS!

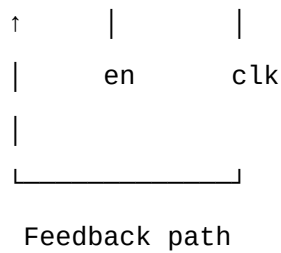
You completely skipped Question 4. Let me answer it for you:

Part A: Hardware Structure

$q \leq q$ synthesizes to a **2-to-1 multiplexer with feedback**:

Input paths:





How it works:

- When $en=1$: MUX selects d → flip-flop captures new data
- When $en=0$: MUX selects q (feedback) → flip-flop recaptures its own output → **holds value**

Part B: Why No Combinational Loop?

Combinational loop (BAD): Output feeds back to input through **pure combinational logic**

```
assign q = some_logic(q); // ← Instant feedback = oscillation!
```

Sequential feedback (GOOD): Output feeds back through a **clocked storage element**

```
always @(posedge clk)
    q <= q; // ← Feedback goes through flip-flop = stable!
```

The **flip-flop breaks the loop** because:

1. Feedback path goes through a **registered element** (the flip-flop itself)
2. Update only happens on **clock edge** (discrete time steps)
3. No continuous combinational path from output back to input

Corrected Understanding: What You Must Know

Priority Chain (CORRECTED):

```
reset > en > hold
```

↑ ↑ ↑

1st 2nd default

NOT: $reset > en > clk > d$ ← This is wrong!

Sensitivity List vs Priority:

- **Sensitivity list:** WHEN the block runs (triggers)
 - `@(posedge clk or posedge reset)` means: run on clock edge OR reset edge
- **Priority logic:** WHAT the block does when it runs (decision tree)
 - `if (reset) ... else if (en) ... else ...`

Clock is NOT in the Priority Chain:

- Clock is a **trigger** (makes the block execute)
- Priority is about **what happens** once triggered

- Clock doesn't compete with reset or enable—it's a different layer
-

Mini Quiz: Prove You Understand

Answer these quick scenarios:

Scenario 1: At time 50ns:

- `clk` = rising edge
- `reset` = 1
- `en` = 1
- `d` = 1

What is `q`? Explain why.

Scenario 2: At time 75ns:

- `clk` = LOW (not an edge)
- `reset` = 0 → 1 (rising edge)
- `en` = 1
- `d` = 1

What happens to `q`? Explain why.

Scenario 3: At time 90ns:

- `clk` = rising edge
- `reset` = 0
- `en` = 0
- `d` = changes from 0 → 1
- `q` = currently 1

What is `q` after this clock edge? Explain why.

Acknowledged - But You Misread the Question

You're right that you were thinking about a **modified** design, but let me show you why that was the wrong interpretation:

The Original Question (Exact Words):

"In our **CURRENT** design, can we ever force `q=1` asynchronously?"

Key phrase: "CURRENT design"

This explicitly asks about the **existing code**, not hypothetical modifications.

Your Interpretation vs My Intent

What I asked: Can the code we wrote do X?

What you answered: Can code (if modified) do X?

Analogy:

- **Me:** "Can your current car seat 7 people?"
 - **You:** "Yes, if I replace it with a minivan"
 - **Me:** "That's not your CURRENT car..."
-

Why This Confusion Happened

I think you saw "can we ever" and interpreted it as "is it possible to make code that..."

But in engineering context:

- **"Can our current design do X?"** = Does the implemented code have this capability?
- **"Can we design something that does X?"** = Is it theoretically possible to create such a design?

These are **very different questions**.

T Flip-Flop - Concept Block (10 min)

What is a T Flip-Flop?

T stands for "Toggle"

Behavior:

- When $T=0$: Output **holds** current value
- When $T=1$: Output **toggles** (flips): $0 \rightarrow 1$ or $1 \rightarrow 0$

Truth Table:

T	Q(current)	Q(next)	Behavior
0	0	0	Hold
0	1	1	Hold
1	0	1	Toggle
1	1	0	Toggle

Equation: $Q_{next} = T \oplus Q_{current}$ (XOR operation)

Why Does This Exist?

Primary use: Frequency dividers and counters

1. **Frequency Division:**

- Connect $T=1$ permanently
- Output toggles every clock cycle
- Result: Output frequency = Clock frequency $\div 2$

2. Binary Counters:

- Chain T flip-flops together
- Each stage divides frequency by 2
- 3 T-FFs $\rightarrow$ 3-bit counter (counts $0 \rightarrow 7$)

3. State Machines:

- Simple two-state toggles (ON/OFF, OPEN/CLOSE)

Real-world example: Your CPU's clock divider uses cascaded T flip-flops to generate different clock domains (e.g., CPU at 3GHz, memory controller at 1.5GHz).

How to Implement T Flip-Flop

Two approaches:

Approach 1: Using D Flip-Flop + XOR (Structural)

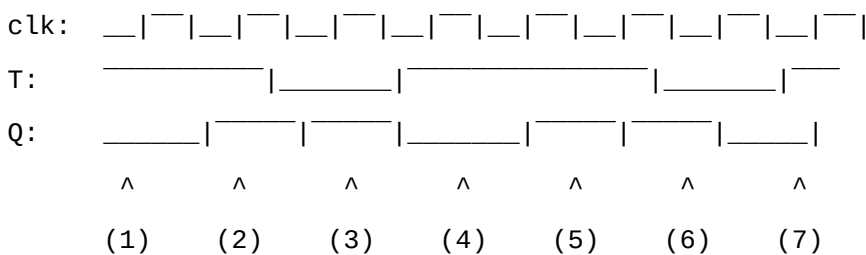
```
// T flip-flop = D flip-flop with feedback through XOR
assign d_input = t ^ q; // XOR current output with T
// Feed this to D flip-flop
```

Approach 2: Behavioral (Direct RTL)

```
always @(posedge clk) begin
    if (t)
        q <= ~q; // Toggle: invert current value
    else
        q <= q; // Hold: keep current value
end
```

We'll use Approach 2 (behavioral) because it's cleaner and more readable.

Timing Diagram



(1) $T=1$: Q toggles $0 \rightarrow 1$

(2) $T=1$: Q toggles $1 \rightarrow 0$

(3) $T=0$: Q holds at 0

- (4) T=1: Q toggles 0→1
- (5) T=1: Q toggles 1→0
- (6) T=0: Q holds at 0
- (7) T=1: Q toggles 0→1

Key observation: When T=1 continuously, Q oscillates at **half the clock frequency**.

Common Mistakes

✗ MISTAKE 1: Forgetting that toggle depends on current state

```
// WRONG
if (t)
    q <= 1; // Always forces to 1, not a toggle!
```

✗ MISTAKE 2: Using blocking assignment

```
// WRONG
if (t)
    q = ~q; // Should be non-blocking <=
```

✗ MISTAKE 3: Inverting T instead of Q

```
// WRONG
if (t)
    q <= ~t; // Should be ~q, not ~t!
```

✓ CORRECT:

```
always @(posedge clk) begin
    if (t)
        q <= ~q; // Toggle: invert Q
    else
        q <= q; // Hold: maintain Q
end
```

Code Block - T Flip-Flop Implementation (10 min)

Design Module

```
// t_ff.v - T Flip-Flop (Toggle Flip-Flop)
module t_ff (
    input wire clk, // Clock signal
    input wire rst, // Asynchronous reset (active HIGH)
    input wire t, // Toggle input
    output reg q // Output
```

```

);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;    // Reset: clear output
    else if (t)
        q <= ~q;      // Toggle: invert current value
    else
        q <= q;        // Hold: maintain current value
    end

endmodule

```

Line-by-Line Explanation

Lines 2-7: Port declarations

- `clk`: Synchronization signal
- `rst`: Async reset (highest priority)
- `t`: Toggle control (when 1, flip the output)
- `q`: Registered output

Line 9: `always @(posedge clk or posedge rst)`

- Triggers on clock rising edge OR reset rising edge
- Asynchronous reset pattern

Lines 10-11: Reset logic

- Priority 1: When `rst=1`, force `q=0`

Lines 12-13: Toggle logic

- Priority 2: When `rst=0` and `t=1`, **invert** current `q`
- `~q` means: if `q=0` → 1, if `q=1` → 0

Lines 14-15: Hold logic

- Priority 3 (default): When `rst=0` and `t=0`, maintain `q`

Testbench

```

// t_ff_tb.v - Testbench for T Flip-Flop
module t_ff_tb;
    reg  clk;
    reg  rst;
    reg  t;

```

```

wire q;

// Instantiate DUT
t_ff uut (
    .clk(clk),
    .rst(rst),
    .t(t),
    .q(q)
);

// Clock generator: 10ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    $dumpfile("t_ff.vcd");
    $dumpvars(0, t_ff_tb);

    $display("Time | clk | rst | t | q");
    $display("-----");
    $monitor("%4t | %b | %b | %b | %b",
        $time, clk, rst, t, q);

    // Initialize
    rst = 1;
    t = 0;
    #10;

    // Release reset
    rst = 0;
    #10;

    // Test 1: Toggle continuously (frequency divider)

```

```

    $display("\n--- Test 1: Continuous Toggle (T=1) ---");
    t = 1;
    #80; // 8 clock cycles - should see Q oscillate

    // Test 2: Hold behavior (T=0)
    $display("\n--- Test 2: Hold (T=0) ---");
    t = 0;
    #40; // 4 clock cycles - Q should stay constant

    // Test 3: Toggle again
    $display("\n--- Test 3: Toggle Again ---");
    t = 1;
    #40; // 4 clock cycles

    // Test 4: Async reset during operation
    $display("\n--- Test 4: Async Reset ---");
    t = 1;
    #15; // Mid-cycle
    rst = 1; #10;
    rst = 0;
    #30;

    $finish;
end

endmodule

```

Expected Behavior

Test 1 (Continuous Toggle):

- T=1 for 8 clock cycles
- Q should flip: $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0$
- **Frequency division:** Q changes at half the clock rate

Test 2 (Hold):

- T=0 for 4 clock cycles
- Q should remain constant (whatever value it had)

Test 3 (Toggle Again):

- T=1 for 4 clock cycles
- Q resumes toggling from current state

Test 4 (Async Reset):

- Assert rst mid-cycle (not on clock edge)
- Q should immediately go to 0
- After reset release with T=1, resume toggling

Compile and Run

```
iverilog -o t_ff t_ff.v t_ff_tb.v
```

```
vvp t_ff
```

```
gtkwave t_ff.vcd
```

In GTKWave:

1. Add all signals: clk, rst, t, q
2. Zoom to Test 1 region
3. **Count:** How many clock cycles per Q transition when T=1?
 - Answer should be: **1 clock cycle** (Q changes every clock edge)
 - Q frequency = clk frequency ÷ 2

Mini Task (5-8 min)

Task: Build a **2-bit binary counter** using T flip-flops.

Requirements:

1. Use TWO T flip-flops (call them q0 and q1)
2. Counter counts: 00 → 01 → 10 → 11 → 00 (repeats)
3. No external T inputs - wire them cleverly using the Q outputs
4. Single clock input drives both flip-flops
5. Async reset to clear counter to 00

Hints:

- q0 (LSB): Should toggle every clock cycle → T0 = ?
- q1 (MSB): Should toggle only when q0=1 → T1 = ?
- Think about the toggle conditions for each bit

Module signature:

```
module counter_2bit (
    input  wire clk,
    input  wire rst,
    output reg  [1:0] q // 2-bit output: {q1, q0}
);
```

Write the complete module and explain your logic for the T inputs.

Submit your counter implementation. I'll review it and explain the pattern for building N-bit counters.

Good Question - Let Me Clarify

You're confused because I said "no external T inputs" but T flip-flops need T signals. Let me explain what I mean.

What "No External T Inputs" Means

External T input would look like this:

```
module counter_2bit (  
    input  wire clk,  
    input  wire rst,  
    input  wire t0,      // ← External control for bit 0  
    input  wire t1,      // ← External control for bit 1  
    output reg  [1:0] q  
);
```

But that's NOT what we want.

Instead, the T signals should be **generated internally** based on the counter's own outputs. The counter should **automatically count** without needing external toggle signals.

How to Build It - Two Approaches

Approach 1: Instantiate Two T Flip-Flops (Structural)

This approach uses the `t_ff` module we just created:

```
module counter_2bit (  
    input  wire clk,  
    input  wire rst,  
    output wire [1:0] q  
);  
  
    wire t0, t1;  // Internal T signals  
  
    // TODO: Generate t0 and t1 based on counting logic  
    assign t0 = ???;  // What should this be?  
    assign t1 = ???;  // What should this be?
```

```
// Instantiate two T flip-flops
t_ff ff0 (
    .clk(clk),
    .rst(rst),
    .t(t0),
    .q(q[0])    // LSB
);

t_ff ff1 (
    .clk(clk),
    .rst(rst),
    .t(t1),
    .q(q[1])    // MSB
);
```

```
endmodule
```

Your task: Figure out what `t0` and `t1` should be.

Approach 2: Behavioral (Single Always Block)

This approach writes the counter behavior directly without instantiating T flip-flops:

```
module counter_2bit (
    input  wire clk,
    input  wire rst,
    output reg  [1:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 2'b00;
    else begin
        // TODO: Implement T flip-flop behavior for each bit
        // Bit 0: Toggle when ???
        // Bit 1: Toggle when ???
    end
end
```

endmodule

Understanding the Counting Pattern

Let's analyze a 2-bit binary counter:

Count	q1	q0	What happens?
0	0	0	Initial state
1	0	1	q0 toggles
2	1	1	q1 toggles
3	1	0	q0 toggles
4	0	0	q1 toggles
5	0	1	q0 toggles
6	1	1	q0 toggles
7	1	0	q1 toggles
8	0	0	q0 toggles
9	0	1	q0 toggles
10	1	1	q1 toggles
11	1	0	q0 toggles
12	0	0	q1 toggles
13	0	1	q0 toggles
14	1	1	q0 toggles
15	1	0	q1 toggles
16	0	0	q0 toggles
17	0	1	q0 toggles
18	1	1	q1 toggles
19	1	0	q0 toggles
20	0	0	q1 toggles
21	0	1	q0 toggles
22	1	1	q0 toggles
23	1	0	q1 toggles
24	0	0	q0 toggles
25	0	1	q0 toggles
26	1	1	q1 toggles
27	1	0	q0 toggles
28	0	0	q1 toggles
29	0	1	q0 toggles
30	1	1	q0 toggles
31	1	0	q1 toggles
32	0	0	q0 toggles
33	0	1	q0 toggles
34	1	1	q1 toggles
35	1	0	q0 toggles
36	0	0	q1 toggles
37	0	1	q0 toggles
38	1	1	q0 toggles
39	1	0	q1 toggles
40	0	0	q0 toggles
41	0	1	q0 toggles
42	1	1	q1 toggles
43	1	0	q0 toggles
44	0	0	q1 toggles
45	0	1	q0 toggles
46	1	1	q0 toggles
47	1	0	q1 toggles
48	0	0	q0 toggles
49	0	1	q0 toggles
50	1	1	q1 toggles
51	1	0	q0 toggles
52	0	0	q1 toggles
53	0	1	q0 toggles
54	1	1	q0 toggles
55	1	0	q1 toggles
56	0	0	q0 toggles
57	0	1	q0 toggles
58	1	1	q1 toggles
59	1	0	q0 toggles
60	0	0	q1 toggles
61	0	1	q0 toggles
62	1	1	q0 toggles
63	1	0	q1 toggles
64	0	0	q0 toggles
65	0	1	q0 toggles
66	1	1	q1 toggles
67	1	0	q0 toggles
68	0	0	q1 toggles
69	0	1	q0 toggles
70	1	1	q0 toggles
71	1	0	q1 toggles
72	0	0	q0 toggles
73	0	1	q0 toggles
74	1	1	q1 toggles
75	1	0	q0 toggles
76	0	0	q1 toggles
77	0	1	q0 toggles
78	1	1	q0 toggles
79	1	0	q1 toggles
80	0	0	q0 toggles
81	0	1	q0 toggles
82	1	1	q1 toggles
83	1	0	q0 toggles
84	0	0	q1 toggles
85	0	1	q0 toggles
86	1	1	q0 toggles
87	1	0	q1 toggles
88	0	0	q0 toggles
89	0	1	q0 toggles
90	1	1	q1 toggles
91	1	0	q0 toggles
92	0	0	q1 toggles
93	0	1	q0 toggles
94	1	1	q0 toggles
95	1	0	q1 toggles
96	0	0	q0 toggles
97	0	1	q0 toggles
98	1	1	q1 toggles
99	1	0	q0 toggles
100	0	0	q1 toggles
101	0	1	q0 toggles
102	1	1	q0 toggles
103	1	0	q1 toggles

```

-----|-----|-----
0    | 0  0  |
1    | 0  1  | q0 toggles
2    | 1  0  | q0 toggles, q1 toggles
3    | 1  1  | q0 toggles
0    | 0  0  | q0 toggles, q1 toggles (wrap)

```

Pattern Recognition:

Bit 0 (q0 - LSB):

- Toggles on **every** clock cycle
- **T0 = 1** (always toggle)

Bit 1 (q1 - MSB):

- Toggles only when $q_0 = 1$
- $T_1 = q_0$ (toggle when LSB is 1)

Visual Timing Diagram

clk: 

q0: 

q1: 

Count: 0 1 2 3 0 1 2 3

Observations:

- q0 changes every clock cycle (period = 2 clocks)
- q1 changes every 2 clock cycles (period = 4 clocks)
- q1 toggles when q0 goes from 1 → 0 (carry propagation)

Your Task Options

Choose ONE approach and implement it:

Option A: Structural (Using `t_ff` instances)

Complete this code:

```
module counter_2bit (
```

```

input  wire clk,
input  wire rst,
output wire [1:0] q
);

wire t0, t1;

assign t0 = ???; // When should bit 0 toggle?
assign t1 = ???; // When should bit 1 toggle?

t_ff ff0 (.clk(clk), .rst(rst), .t(t0), .q(q[0]));
t_ff ff1 (.clk(clk), .rst(rst), .t(t1), .q(q[1]));

endmodule

```

Option B: Behavioral (Direct implementation)

Complete this code:

```

module counter_2bit (
    input  wire clk,
    input  wire rst,
    output reg [1:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 2'b00;
    else begin
        // Implement T flip-flop toggle logic
        if (???) // When should q[0] toggle?
            q[0] <= ~q[0];
        else
            q[0] <= q[0];

        if (???) // When should q[1] toggle?
            q[1] <= ~q[1];
        else
            q[1] <= q[1];
    end
end

```

```
        end
    end
```

```
endmodule
```

Hints for the Conditions

For q[0] (LSB):

- Should it toggle every clock cycle?
- What should the condition be?

For q[1] (MSB):

- Should it toggle every clock cycle?
 - Or only when something specific happens with q[0]?
 - Think: In binary counting, when does a higher bit flip?
-

Also Write a Testbench

```
module counter_2bit_tb;
    reg clk, rst;
    wire [1:0] q;

    counter_2bit uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("counter_2bit.vcd");
        $dumpvars(0, counter_2bit_tb);

        $display("Time | rst | q[1:0] | Decimal");
        $display("-----");
```

```
$monitor("%4t | %b | %b | %d",
         $time, rst, q, q);

rst = 1; #10;
rst = 0;
#100; // Let it count through multiple cycles

$finish;
end

endmodule
```

What I Want From You

1. **Choose an approach** (Structural or Behavioral)
2. **Fill in the ??? parts** with your logic
3. **Explain your reasoning:**
 - Why did you choose those conditions?
 - How does it implement binary counting?
4. **Show me the complete code** (module + testbench)

Don't guess. Think through the counting pattern step by step.

Submit your implementation and explanation.